Ledbetter Data Pump (LDP)

Phase 1 Implementation Contract

Ready for coding agents and engineers

Version 1.0 • November 2026

Ledbetter Data Pump (LDP)

Phase 1 Implementation Contract

Version 1.0

Date: November 2026

Status: Ready for coding agents and engineers

This document is the precise implementation contract for **Phase 1**.

It supplements the Architecture White Paper and the Failure Modes Addendum.

An implementer (human or LLM) should be able to produce a working Phase 1 system from these three documents alone.

1. Phase 1 Scope

In scope

• YAML parsing + semantic validation

• Arrow `RecordBatch`-based data flow

• Partitioned source reading (range partitions)

• Worker pool with bounded channels

• Basic transforms: filter + derive (restricted expression subset)

• Checkpointing (atomic JSON state file)

• Postgres source connector

• Parquet sink connector

• CSV sink (basic)

• Structured JSON logging

• CLI binary

• Core edge-case handling defined for Phase 1 in the addendum

Out of scope for Phase 1

• Snowflake / SQL Server / Oracle

• Conditional split

• Atomic staging/swap

• Full DLQ + error budgets (basic row isolation only)

• Watermarks / incremental

• Graceful shutdown

• Python bindings

• DataFusion

2. Crate Layout and Responsibilities

```
ldp/
■■■ Cargo.toml                # workspace
■■■ ldp-config/               # YAML schema, validation, semantic checks, ExecutionPlan
■■■ ldp-core/                 # DataChunk, worker pool, channels, checkpointing, transforms
■■■ ldp-arrow/                # Type mapping helpers, batch utilities
■■■ ldp-connectors/
■   ■■■ postgres/             # Postgres Source
■   ■■■ parquet/              # Parquet Sink (+ basic CSV)
■■■ ldp-cli/                  # Binary: ldp run pipeline.yaml
■■■ README.md
```

Dependency direction

`ldp-cli` → `ldp-config` + `ldp-core` + connectors

`ldp-core` → `ldp-arrow`

Connectors → `ldp-core` + `ldp-arrow`

No cycles.

3. Core Rust Types and Traits

3.1 Data Model

```
// ldp-core/src/chunk.rs
use arrow::record_batch::RecordBatch;

#[derive(Debug, Clone)]
pub struct DataChunk {
    pub chunk_id: u64,
    pub batch: RecordBatch,
}

impl DataChunk {
    pub fn num_rows(&self) -> usize {
        self.batch.num_rows()
    }
}
```

```
// ldp-core/src/partition.rs
#[derive(Debug, Clone)]
pub struct Partition {
    pub id: u64,
    pub start: Option<i64>,   // inclusive, for range
    pub end: Option<i64>,     // exclusive
}
```

3.2 Execution Plan

```
// ldp-config/src/plan.rs
#[derive(Debug, Clone)]
pub struct ExecutionPlan {
    pub pipeline_name: String,
    pub max_threads: usize,
    pub partitions: Vec<Partition>,
    pub stages: Vec<Stage>,
    pub state_file: String,
}

#[derive(Debug, Clone)]
pub enum Stage {
    Source { id: String, /* connector config */ },
    Filter { id: String, input: String, predicate: Expr },
    Derive { id: String, input: String, fields: Vec<DeriveField> },
    Sink { id: String, input: String, /* connector config */ },
```

```
}
```

3.3 Traits

```
// ldp-core/src/traits.rs
use async_trait::async_trait;
use crate::{DataChunk, Partition, LdpError};

#[async_trait]
pub trait Source: Send + Sync {
    async fn read_partition(&self, partition: &Partition) -> Result<DataChunk, LdpError>;
}

#[async_trait]
pub trait Sink: Send + Sync {
    async fn write_chunk(&self, chunk: &DataChunk) -> Result<(), LdpError>;
    async fn finalize(&self) -> Result<(), LdpError>;
}

pub trait Transform: Send + Sync {
    fn apply(&self, chunk: DataChunk) -> Result<DataChunk, LdpError>;
}
```

3.4 Error Type

```
// ldp-core/src/error.rs
#[derive(Debug, thiserror::Error)]
pub enum LdpError {
    #[error("configuration error: {0}")]
    Config(String),
    #[error("source error: {0}")]
    Source(String),
    #[error("transform error: {0}")]
    Transform(String),
    #[error("sink error: {0}")]
    Sink(String),
    #[error("checkpoint error: {0}")]
    Checkpoint(String),
    #[error("expression error on row: {0}")]
    Expression(String),
    #[error(transparent)]
    Other(#[from] anyhow::Error),
}
```

3.5 Checkpoint

```
// ldp-core/src/checkpoint.rs
use serde::{Deserialize, Serialize};
use std::collections::HashSet;

#[derive(Debug, Serialize, Deserialize, Default)]
pub struct PipelineState {
    pub completed_chunks: HashSet<u64>,
}

impl PipelineState {
    pub fn load(path: &str) -> Result<Self, LdpError> { /* atomic read */ }
    pub fn save(&self, path: &str) -> Result<(), LdpError> { /* write tmp + rename */ }
    pub fn is_completed(&self, id: u64) -> bool { self.completed_chunks.contains(&id) }
    pub fn mark_completed(&mut self, id: u64) { self.completed_chunks.insert(id); }
}
```

4. Phase 1 Expression Subset

Supported in `filter` conditions and `derive` expressions.

Allowed

• Column references: `amount`, `status`

• Literals: numbers, strings (`'CANCELLED'`), booleans, `null`

• Comparisons: `=`, `!=`, `>`, `>=`, `<`, `<=`

• Logical: `AND`, `OR`, `NOT` (parentheses supported)

• Arithmetic: `+`, `-`, `*`, `/`

• Null checks: `IS NULL`, `IS NOT NULL`

• Simple functions: `coalesce(a, b)`, `now()` (returns timestamp)

Explicitly Not Supported in Phase 1

• `IN` lists (add in Phase 2)

• String functions (`lower`, `substring`, etc.)

• Aggregates

• Subqueries

• User-defined functions

Evaluation Rules

• Use Arrow compute kernels where possible.

• Null propagation follows SQL three-valued logic for comparisons.

• Division by zero or type error on a row → `LdpError::Expression` → row isolation (drop row + log in Phase 1; full DLQ in Phase 2).

• All expressions are compiled once per stage, not per row.

Example

```
condition: "status != 'CANCELLED' AND amount > 0 AND updated_at IS NOT NULL"
expression: "amount * 1.0825"
```

5. Type Mapping Rules (Phase 1)

Centralized in `ldp-arrow`.

Rules

• Unknown Postgres type → error at planning time.

• Nullability is preserved.

• No silent truncation of decimals.

• Strings are UTF-8 only in Phase 1.

6. YAML Schema Outline (Phase 1)

```
version: "1.0"                  # required, must be "1.0"
pipeline:
  name: string                  # required
```

```
  settings:
    max_threads: integer          # default 8
    state_file: string            # default ".ldp/<name>.json"
    max_batch_rows: integer       # default 65536
    max_batch_bytes: integer      # default 128_000_000
  resources:
    connections:
      - name: string
        type: "postgres"          # only postgres in Phase 1
        url: string               # env:VAR or postgres://...
  dataflow:
    sources:
      - id: string
        connection: string
        query: string             # required
        partitioning:
          type: "range" | "none"
          column: string          # required if range
          chunks: integer         # required if range
    transformations:              # optional
      - id: string
        input: string
        type: "filter" | "derive"
        condition: string         # filter only
        fields:                   # derive only
          - name: string
            expression: string
    targets:
      - id: string
        input: string
        type: "parquet" | "csv"
        path: string
        compression: "zstd" | "snappy" | "none"   # parquet
```

Semantic validation (must pass before execution)

• All `input` references resolve

• No cycles

• Partition column is present in the spirit of the query (documented assumption in Phase 1)

• Exactly one source in Phase 1 (simplifies planning)

7. Worker Pool Sketch (Phase 1)

```
// Simplified flow
pub async fn run(plan: ExecutionPlan, source: Arc<dyn Source>, sink: Arc<dyn Sink>) -> Result<(), LdpError> {
    let mut state = PipelineState::load(&plan.state_file)?;
    let (tx, rx) = bounded::<DataChunk>(plan.max_threads * 2);

    // Spawn workers that pull partitions, skip completed, read → transform → send
    // Main task receives from rx and writes to sink, then marks checkpoint

    // On full success: state.save()
}
```

• Use `rayon` or `tokio` tasks for workers (tokio preferred because of async Source).

• Bounded channel provides backpressure.

• Checkpoint is updated only after successful sink write.

8. Definition of Done — Phase 1

Phase 1 is complete when all of the following are true:

1. `ldp run pipeline.yaml` executes a Postgres → Parquet job with range partitioning.

2. Filter and derive stages using the Phase 1 expression subset work and are vectorized where possible.

3. Restarting a failed job skips already-completed chunks (verified by killing the process mid-run).

4. Empty source or fully-filtered result produces a successful run with 0 rows and a valid (possibly empty) Parquet file.

5. Basic memory limits (`max_batch_rows` / `max_batch_bytes`) are enforced.

6. Atomic checkpoint writes (tmp + rename) are used.

7. Semantic validation rejects obviously invalid YAML before any query is issued.

8. Structured JSON logs are emitted for chunk start/complete and final summary.

9. At least one integration test: real Postgres (via testcontainers or docker) → Parquet, including a restart test.

10. `cargo test` and `cargo clippy` pass cleanly.

9. Recommended Crates (Phase 1)

• `arrow`, `parquet` — data + sink

• `serde`, `serde_yaml` — config

• `tokio`, `async-trait` — async

• `sqlx` (postgres feature) — source

• `thiserror`, `anyhow` — errors

• `tracing` + `tracing-subscriber` (json) — logging

• `crossbeam-channel` or `tokio::sync::mpsc` — channels

10. Notes for Implementers

• Prefer explicit types over heavy generics in Phase 1.

• Keep the expression evaluator simple and correct; performance can be improved later.

• Write the Postgres connector to be partition-aware (`WHERE id >= $1 AND id < $2`).

• Parquet sink should write one file per chunk or a single file with row groups — either is acceptable in Phase 1 (document the choice).

• Do not implement features marked Phase 2 even if they seem easy.

End of Phase 1 Implementation Contract

This document + the Architecture White Paper + the Failure Modes Addendum form the complete package required to begin implementation.